

PROCESOS DE INGENIERÍA DEL SOFTWARE 2

Tema 15 - Servicios Web e IA (MCP)



UNIVERSIDAD
DE ALMERÍA

Grupo 6

Juan Francisco Escobosa Brocal

Abel Rodríguez Gutiérrez

Víctor Vallejo Uroz

ÍNDICE DE CONTENIDOS

	Pág.
1. Fundamentos Modernos	3
1.1. Arquitecturas Orientadas a Servicios (SOA) y Microservicios.....	3
1.2. APIs REST (APIs Resilientes).....	4
2. Model Context Protocol (MCP)	5
2.1. Anthropic y su Estándar.....	5
2.2. ¿Cómo puede “leer” y “escribir” un LLM en herramientas externas a través de un estándar?	6
3. Tool Use.	7
3.1. ¿Qué es un Tool Use?.....	7
3.2. “Tool Use” vs servidor MCP	7
4. Metodología de Construcción de un Servidor MCP.	8
4.1. Fases del Desarrollo Técnico y Ciclo de Vida.	9
4.2. Ingeniería de Primitivas MCP.	10
4.3. Patrones de Comunicación y Capa de Transporte.	11
4.4. Resiliencia, Seguridad y Gobernanza Perimetral.	11
5. Auditoría de Calidad: Estándares ISO Aplicados.	12
5.1. ISO/IEC 25010: Ingeniería de Calidad del Producto de Software.	12
5.2. ISO/IEC 42001: Estándar Mundial de Gestión de IA.	12
5.3. Sinergia Técnica para la Certificación.	13
6. Bibliografía y Referencias.	14

1. Fundamentos Modernos

La evolución desde el monolito de código hasta los sistemas distribuidos no responde a una moda o tendencia, sino a la necesidad de gestionar la complejidad cognitiva y escalada independiente.

La complejidad cognitiva es la cantidad de esfuerzo mental que un desarrollador necesita para comprender, mantener y predecir el comportamiento de un sistema. En un monolito, con el tiempo las fronteras entre módulos se difuminan. Por ejemplo, una modificación en un módulo de facturación puede provocar efectos secundarios en un módulo de logística debido a dependencias ocultas. Además un desarrollador no puede especializarse. Para tocar una línea de código, debe entender la totalidad de la aplicación. Esto ralentiza el tiempo de salida al mercado y aumenta la probabilidad de errores humanos.

La solución que se ha adoptado es el uso de microservicios. Al fragmentar el sistema en Contextos delimitados reducimos la carga mental. Así, el desarrollador solo necesita entender la API de los demás servicios y la lógica interna del suyo.

1.1. Arquitecturas Orientadas a Servicios (SOA) y Microservicios

La arquitectura orientada a servicios nació como solución a la falta de aislamiento de los sistemas legacy. Su principal objetivo era la reutilización y sus características principales incluían:

- **Protocolos Estrictos:** Principalmente basados en **SOAP** (Simple Object Access Protocol), que requiere el uso de WSDL (archivos XML de definición de servicio) para establecer contratos rígidos.
- **Interoperabilidad Empresarial:** Permite que, por ejemplo, un sistema en COBOL hable con uno en Java, pero a costa de una gran sobrecarga de computación debido al procesamiento de XML.
- **Inteligencia Centralizada (ESB):** El **Enterprise Service Bus** es el corazón de SOA. El bus no solo transporta datos; los transforma, los traduce y aplica reglas de negocio.

Riesgo Crítico: En SOA, el ESB a menudo se convierte en un "agujero negro" de lógica. Si el bus cae, la empresa entera se detiene. Es lo que llamamos **Acoplamiento de Infraestructura**.

Debido a este último riesgo, se vio que el bus central era más problemático de lo esperado así que las arquitecturas orientadas a servicios evolucionaron naturalmente a microservicios, que es prácticamente lo mismo pero eliminando la dependencia del bus central.

La principales diferencias son:

Smart Endpoints, Dumb Pipes: A diferencia de SOA, aquí la red solo mueve bits, generalmente vía REST/JSON o gRPC. La lógica de transformación y negocio reside exclusivamente dentro del microservicio.

Descentralización de Datos: En SOA, los servicios suelen compartir una base de datos gigante. En Microservicios, **cada servicio tiene su propia base de datos**. Esto elimina el acoplamiento a nivel de esquema de datos, pero introduce el reto de la **consistencia eventual**.

Gobernanza Descentralizada: Cada equipo puede elegir el stack tecnológico (lenguaje, base de datos) que mejor se adapte a su problema específico.

1.2. APIs REST (APIs Resilientes)

El diseño resiliente nace de desmentir ocho suposiciones falsas que los desarrolladores suelen cometer:

1. La red es fiable.
2. La latencia es cero.
3. El ancho de banda es infinito.
4. La red es segura.
5. La topología no cambia.
6. Hay un solo administrador.
7. El coste de transporte es cero.
8. La red es homogénea.

Para superar la teoría básica, una arquitectura de integración moderna debe implementar los siguientes tres patrones de forma obligatoria:

A. Patrón Circuit Breaker (Disyuntor)

Evita el **fallo en cascada**. Si un servicio remoto empieza a fallar, el disyuntor "salta" para proteger al sistema local de quedarse sin recursos esperando respuestas que no llegarán.

- **Estado CERRADO:** Las peticiones fluyen. Se monitorizan los fallos.
- **Estado ABIERTO:** Se alcanza el umbral de error. Las peticiones fallan inmediatamente por lo que el servicio remoto tiene tiempo de recuperarse.
- **Estado SEMI-ABIERTO:** Tras un tiempo de espera, se permite que unas pocas peticiones pasen. Si tienen éxito, el circuito se cierra de nuevo.

B. Patrón Bulkhead (Mamparos)

Aísla los recursos para que un fallo en un área no hunda todo el sistema.

- **Implementación:** En lugar de tener un único pool de hilos para todas las llamadas salientes, se asignan pools separados por servicio.
- **Efecto:** Si el servicio de "Generación de PDFs" se bloquea y consume sus 10 hilos asignados, el servicio de "Autenticación" sigue funcionando porque tiene sus propios hilos reservados.

C. Patrón Retry con Exponential Backoff y Jitter

Reintentar una petición es útil para fallos temporales de red, pero puede ser destructivo si no se hace con inteligencia.

1. **Exponential Backoff:** El tiempo por intento aumenta exponencialmente (1s, 2s, 4s, 8s...) para no saturar al receptor.
2. **Jitter (Variación):** Se añade un componente aleatorio al tiempo de espera.
 - *Sin Jitter:* 1000 clientes fallan a la vez → los 1000 reintentan exactamente a los 2 segundos → El servidor vuelve a caer por el pico de tráfico.
 - *Con Jitter:* Los clientes intentan entre los 1.5s y 2.5s, distribuyendo la carga de forma uniforme.

Timeouts Dinámicos

Toda llamada a una API debe tener un límite de tiempo. Un diseño resiliente no usa el mismo timeout para todo:

- Una consulta a caché: **50ms**.
- Una consulta a base de datos: **500ms**.
- Una llamada a un servicio externo de terceros: **2s**.
- **Fail-fast:** Es preferible fallar en 100ms que dejar al usuario esperando 30 segundos para finalmente darle un error.

Rate Limiting y Throttling

Protege a la API de ser sobrepasada por sus consumidores (ya sean humanos o agentes de IA).

- **Token Bucket:** Un algoritmo que permite ráfagas de tráfico pero mantiene una media de peticiones constante.
- **Leaky Bucket:** Suaviza los picos de tráfico procesándolos a una velocidad constante.

Relación con MCPs

Cuando un modelo de IA (agente) empieza a usar herramientas a través de APIs, el comportamiento se vuelve menos predecible que el de un usuario humano. La IA puede generar cientos de llamadas en segundos si entra en un bucle lógico.

Resiliencia ante la IA: Si el modelo de IA invoca una herramienta que está lenta, los patrones de **Circuit Breaker** y **Timeout** evitan que el agente bloquee el hilo de ejecución principal.

Contexto de Error: Una API resiliente devuelve errores semánticos que un modelo de IA puede entender (ej. "Estoy saturado, reinténtalo en 5 segundos" vs un error genérico).

Gobernanza: Los protocolos como MCP se apoyan en estas APIs robustas para asegurar que el intercambio de contexto entre el modelo y los datos sea estable, seguro y eficiente.

2. Model Context Protocol (MCP)

Un servidor **Model Context Protocol (MCP)** es una implementación de un protocolo de comunicación estandarizado, concebido para actuar como un *gateway* entre un Modelo de Lenguaje Grande (LLM) y un conjunto de herramientas o servicios externos. Técnicamente, su rol es abstraer la complejidad inherente a los sistemas externos (como bases de datos, APIs o sistemas de gestión) y presentarlos al LLM en un formato de contexto uniforme y descriptivo, generalmente basado en esquemas de datos o estructuras de metadatos predefinidas. Esto permite que el LLM pueda razonar sobre la existencia y capacidad de estas herramientas sin requerir conocimiento de sus detalles de implementación subyacentes, logrando así una integración desacoplada y escalable de capacidades externas en el proceso de inferencia del modelo.

2.1. Anthropic y su Estándar

La estrategia clásica para incluir capacidades de inteligencia artificial en un software es vincular directamente los modelos de lenguaje (LLMs) a las bases de datos o APIs de la aplicación. Este

enfoque crea una deuda técnica de complejidad $O(N \times M)$ en un ecosistema que incluye numerosos modelos cognitivos (por ejemplo, Gemini, OpenAI y Claude) y diversas áreas de infraestructura (por ejemplo, calendario o base de datos).

- Acoplamiento fuerte: Para cada inteligencia artificial, el acoplamiento aumenta.
- Infracción de SOLID: El principio de responsabilidad única (SRP) y el principio de open/close (OCP) se ven comprometidos. La aplicación tiene que ser modificada cada vez que un proveedor de inteligencia artificial o una herramienta externa (por ejemplo, Google Calendar) realiza cambios en sus reglas. Esto aumenta la posibilidad de que el sistema se caiga.

La estrategia técnica central consiste en aplicar el diseño guiado por el dominio a la infraestructura de IA, aislando cada sistema core de la empresa en su propio **Servidor MCP**. Esta arquitectura exige una separación estricta de responsabilidades en la capa de red, resultando en servidores independientes clasificados por su frontera lógica. En un ecosistema empresarial estándar, la estructura se podría dividir en:

- **Servidor MCP de Base de Datos (Persistencia):** Gestiona exclusivamente la conexión, lectura y escritura en los sistemas de almacenamiento, aislando la lógica de las consultas.
- **Servidor MCP de Servicios de Terceros (APIs Externas):** Actúa como proxy exclusivo para interactuar con herramientas corporativas externas.
- **Servidor MCP de Sistemas de Archivos (Conocimiento Local):** Gestiona el acceso seguro a los repositorios documentales privados o bases de conocimiento internas de la organización.

2.2. ¿Cómo puede “leer” y “escribir” un LLM en herramientas externas a través de un estándar?

Para que un LLM pueda “leer” o “escribir” en herramientas externas sin ser un sistema caótico, el estándar MCP actúa como un contrato de interfaz visual. Para que una IA sea útil, no solo debe saber “hablar”, sino también debe saber **interactuar**. Para eso existe el estándar MCP.

La IA no puede entrar en las carpetas o bases de datos por sí sola ya que para ella el mundo exterior es invisible. Cuando la IA debe de analizar bases de datos o necesita algún tipo de dato, le pide al servidor MCP que vaya a buscarlo. El servidor actúa como un traductor, ya que su función es entrar en la herramienta (como Google Drive o YouTube), extrae la información y la convierte en texto plano o markdown para que la IA pueda entenderlo y analizarlo, y ese es el resultado.

¿Qué ocurre cuando quiere “escribir”? En este caso, la IA no tiene manos para pulsar botones o enviar correos. Para eso, el estándar MCP le permite delegar la tarea. Para ello, el servidor le presenta a la IA ciertas herramientas. Si se desea enviar un email, la IA rellena una orden de trabajo técnica con el mensaje que ha redactado. Gracias a esa ejecución, el servidor recibe la orden y él es el que ejecuta la acción.

3. Tool Use

3.1. ¿Qué es Tool Use?

“Tool Use” o “Function Calling” trata sobre la capacidad de una IA de actuar y dejar de ser un generador de texto. ¿Cómo funciona el ciclo del “Tool Use”?

El proceso no es magia; es una conversación técnica de ida y vuelta que sigue estos pasos:

1. Detección de necesidad: Tú le pides algo a la IA (ej: "¿Cuál es la cotización actual de Apple?"). La IA sabe que su entrenamiento es antiguo y que necesita un dato en tiempo real.
2. Selección de la herramienta: La IA mira su "caja de herramientas" (proporcionada por el desarrollador) y elige la más adecuada (ej: una herramienta llamada `get_stock_price`).
3. Generación de la "Comanda" (Arguments): La IA genera un código estructurado (generalmente un JSON) que dice: ejecutar: `get_stock_price`, parámetros: { "empresa": "AAPL" }. Ojo: La IA no ejecuta el código, solo escribe la orden.
4. Ejecución Externa: Tu sistema (o un Servidor MCP) recibe esa orden, hace la llamada real a internet y obtiene el dato (ej: "180.50€").
5. Interpretación Final: El sistema le devuelve ese dato a la IA. La IA lo lee, lo entiende y finalmente te responde a ti: "La cotización actual de Apple es de 180.50€".

3.2. Diferenciación Arquitectónica: Tool Use frente a Servidor MCP

A pesar de que en la literatura divulgativa del sector ambos términos suelen emplearse de manera intercambiable, la capacidad de Tool Use y el protocolo MCP (Model Context Protocol) constituyen conceptos diametralmente distintos que operan en niveles separados dentro de la arquitectura del software. Su distinción radica en la dicotomía entre la capacidad de razonamiento del agente y la infraestructura de integración subyacente.

3.2.1. La Capa Cognitiva frente a la Capa de Infraestructura

Tool Use (La Capacidad Cognitiva): Define la competencia algorítmica e intrínseca de un Modelo Fundacional de Lenguaje (LLM) para interrumpir su generación estocástica de texto, estructurar un conjunto de datos (habitualmente en formato JSON) y emitir una directiva para la ejecución de una función externa. Constituye el razonamiento operativo del modelo.

El Servidor MCP representa el protocolo técnico y la capa de interfaz que normaliza la exposición de las capacidades de los sistemas empresariales frente al LLM. MCP actúa como un traductor universal que estandariza el contrato de comunicación.

El principal problema arquitectónico que resuelve MCP es la escalabilidad de las integraciones. En un paradigma de integración directa (tradicional), conectar un ecosistema de N modelos de lenguaje (ej. Claude, GPT-4, Gemini) con M herramientas corporativas (ej. GitHub, Jira, PostgreSQL) exige desarrollar y mantener integraciones personalizadas para cada combinación, resultando en una complejidad de $O(N \times M)$.

El estándar MCP introduce una arquitectura en estrella basada en el Principio de Inversión de

Dependencias (DIP). Al implementar este protocolo, cada modelo solo necesita comprender el estándar MCP, y cada herramienta solo requiere un único Servidor MCP. Esto reduce la complejidad de la red a $O(N+M)$. El modelo se vuelve completamente agnóstico respecto a la implementación real del backend de la herramienta.

En una arquitectura MCP, la responsabilidad de ejecución está estrictamente delimitada:

1. El Modelo (Tool Use) "solicita" el uso de una herramienta basándose en la descripción semántica que el servidor le ha proporcionado previamente.
2. El Cliente MCP enruta esta petición estructurada.
3. El Servidor MCP es el único actor que posee la lógica de negocio y las credenciales para ejecutar la acción real contra la infraestructura, devolviendo únicamente el resultado de dicha operación al modelo.

3.2.2. Soberanía del Dato y Seguridad Perimetral

Desde el punto de vista de la ciberseguridad corporativa, la integración directa de APIs con proveedores de IA externos supone un riesgo de fuga de información. El Servidor MCP actúa como un centinela perimetral (firewall de contexto).

En lugar de exponer una base de datos entera o claves de API al proveedor del LLM, el Servidor MCP retiene el control absoluto sobre qué recursos son visibles. Evalúa la petición del modelo, ejecuta la consulta internamente y devuelve al LLM única y exclusivamente el contexto estrictamente necesario para resolver la tarea del usuario, garantizando así la soberanía de los datos sensibles de la organización.

La construcción de un servidor MCP representa la implementación práctica de la capa de infraestructura que permite la interoperabilidad entre modelos de lenguaje y sistemas externos. Para un entorno académico, es fundamental estructurar este proceso no solo como una guía de codificación, sino como una metodología de diseño de sistemas basada en la exposición de capacidades y la seguridad perimetral.

4. Metodología de Construcción de un Servidor MCP.

La implementación de un servidor bajo el estándar *Model Context Protocol* (MCP) trasciende la mera codificación de un intermediario (*middleware*); constituye un riguroso ejercicio de ingeniería de software orientado a definir las fronteras de un dominio de datos y exponer sus capacidades de forma segura frente a agentes de Inteligencia Artificial.

El desarrollo de estos servidores se fundamenta en el uso de los SDKs (*Software Development Kits*) oficiales, programados mayoritariamente en lenguajes con tipado estricto como TypeScript o ecosistemas orientados a datos como Python. El valor arquitectónico de estos SDKs radica en su capacidad para abstraer por completo la complejidad del protocolo de mensajería asíncrona subyacente (basado en JSON-RPC 2.0). Esto permite a los ingenieros aplicar los principios del Diseño Guiado por el Dominio (*Domain-Driven Design*), focalizando el esfuerzo en la lógica de negocio y en la semántica de las herramientas, en lugar de gestionar la serialización de datos, la cabecera de las tramas o el control de la red.

4.1. Fases del Desarrollo Técnico.

El ciclo de vida de un servidor MCP comienza con un proceso de conexión estrictamente regulado. En una arquitectura de estas características, un Modelo de Lenguaje (LLM) jamás interactúa directamente con el servidor. Es el Cliente MCP (integrado en la aplicación anfitriona u Host) quien debe establecer, asegurar y mantener el estado de la conexión. Antes de que se procese cualquier operación de lectura o escritura, ambos extremos deben acordar un contrato operativo seguro.

4.1.1. El proceso de *Handshake* (Negociación de Capacidades).

El *handshake* (apretón de manos) es el primer y más crítico intercambio de mensajes del protocolo. Funciona como un mecanismo de descubrimiento y validación de versiones:

1. **Petición de inicialización:** El Cliente MCP emite una solicitud declarando la versión máxima del protocolo que soporta y su propia matriz de capacidades (por ejemplo, notificando si el cliente soporta paginación de listas de herramientas o capacidades avanzadas de muestreo).
2. **Respuesta del Servidor:** El servidor procesa la solicitud, selecciona la versión del protocolo más alta que ambos compartan (garantizando así la retrocompatibilidad) y responde declarando su propio conjunto de características. En este paso, el servidor especifica explícitamente si su arquitectura expone Recursos (*Resources*), Herramientas (*Tools*), o Plantillas (*Prompts*).
3. **Confirmación:** Una vez que el cliente recibe y acepta la configuración del servidor, envía una notificación final confirmando que la conexión está abierta y lista para enrutar tráfico.

Esta negociación simétrica elimina la incertidumbre en sistemas distribuidos, evitando excepciones en tiempo de ejecución causadas por un LLM intentando invocar primitivas que un servidor específico no implementa.

4.1.2. Configuración de Metadatos y Trazabilidad.

Durante el proceso de inicialización, junto con la matriz de capacidades técnicas, el desarrollador debe inyectar la identidad del servidor definiendo parámetros críticos como el nombre y la versión del componente.

Aunque esta parametrización pueda parecer trivial desde un punto de vista puramente funcional, constituye un pilar fundamental para la gobernanza corporativa y la arquitectura de microservicios:

1. **Enrutamiento Determinista:** En un ecosistema empresarial, el Host puede estar conectado simultáneamente a decenas de servidores MCP (uno para la base de datos, otro para Jira, otro para el calendario). Los metadatos permiten al Cliente MCP construir un mapa de ruteo preciso; si el LLM solicita consultar facturas, el cliente sabe exactamente hacia qué componente redirigir la trama JSON-RPC.
2. **Trazabilidad y Depuración Operativa:** Ante una latencia anómala o un fallo en la ejecución, los metadatos permiten a los sistemas de monitorización identificar de forma unívoca qué microservicio originó el cuello de botella.

3. **Auditoría Normativa:** Alineado con los estándares ISO mencionados anteriormente, el registro de la versión exacta del servidor que procesó una solicitud de un LLM permite a los auditores de seguridad reconstruir incidentes y verificar si, en el momento de una modificación de datos, el servidor MCP operaba con un parche de seguridad actualizado.

4.2. Ingeniería de Primitivas MCP (El Núcleo Funcional)

Las primitivas representan la superficie de exposición del servidor MCP. En el diseño de sistemas orientados a agentes cognitivos, la calidad de estas definiciones determina la fiabilidad y precisión de la Inteligencia Artificial. No se trata simplemente de exponer funciones aisladas, sino de crear un lenguaje de capacidades estructurado que el modelo algorítmico pueda interpretar y razonar.

4.2.1. Modelado de Recursos (Resources)

Los recursos constituyen los activos de información de solo lectura que el servidor pone a disposición del modelo para enriquecer su contexto operativo. Al igual que en la arquitectura de la *World Wide Web*, cada recurso se identifica de forma inequívoca mediante un esquema de URI único, lo que permite a la aplicación anfitriona realizar peticiones deterministas.

Desde el punto de vista arquitectónico, los servidores pueden exponer catálogos de recursos estáticos o, alternativamente, generar plantillas dinámicas donde la URI contiene variables inyectables. Esta flexibilidad faculta al modelo para solicitar datos específicos en tiempo de ejecución. Asimismo, resulta imperativo asociar cada recurso con un tipo MIME concreto. En la ingeniería de sistemas integrados con modelos de lenguaje, se prioriza el uso de formatos estructurados pero legibles, fundamentalmente Markdown, dado que su jerarquía semántica facilita al modelo la distinción entre metadatos y datos críticos, optimizando así el procesamiento dentro de su ventana de contexto.

4.2.2. Diseño de Herramientas (Tools) y Contratos JSON.

Las herramientas representan el componente activo de la arquitectura, otorgando al modelo la capacidad de ejecutar acciones y alterar el estado del sistema subyacente. Su diseño se rige por el principio de contrato estricto, materializado a través de esquemas de validación estandarizados (JSON Schema). Este esquema desempeña una doble función indispensable: por un lado, instruye al modelo sobre la estructura exacta y los tipos de datos requeridos para la invocación; por otro, actúa como un mecanismo de seguridad perimetral. El SDK del protocolo utiliza este esquema para validar sintácticamente los argumentos generados por la IA antes de ceder el control a la lógica de negocio, mitigando así el riesgo de inyecciones o manipulaciones no deseadas.

Además de la validación estructural, la ingeniería de descripciones semánticas adquiere una relevancia crítica en este componente. A diferencia de las interfaces de programación tradicionales, donde la firma del método es suficiente para el desarrollador humano, en el estándar MCP la descripción en lenguaje natural equivale a código funcional. El modelo de lenguaje selecciona la herramienta basándose exclusivamente en la interpretación de esta descripción. Por consiguiente, una especificación rigurosa en el servidor debe detallar explícitamente no solo el propósito de la herramienta, sino también sus efectos secundarios, restricciones operativas y precondiciones necesarias.

4.2.3. Plantillas de Prompts

Las plantillas de interacción predefinidas, conocidas como *prompts*, conforman el tercer pilar funcional del servidor. A diferencia de las instrucciones estáticas incrustadas en el código cliente, estas

primitivas posibilitan la inyección dinámica de contexto empresarial y la estandarización de flujos de trabajo complejos. Su ubicación centralizada en el servidor MCP garantiza una estricta separación entre la lógica de dominio y la capa de presentación. Al unificar el conocimiento sobre cómo debe abordarse una tarea específica directamente en el servidor de infraestructura, las organizaciones pueden actualizar sus directrices operativas de manera global sin requerir modificaciones en las múltiples aplicaciones o interfaces que consumen el servicio.

4.3. Patrones de Comunicación y Capa de Transporte

La elección de la capa de transporte define la topología de red del ecosistema MCP. Para entornos de desarrollo integrados o aplicaciones de ejecución local, se emplea mayoritariamente el transporte a través de flujos estándar del sistema operativo (Stdio). En este paradigma, el servidor opera como un subproceso local aislado, garantizando latencias mínimas y eliminando la necesidad de exponer puertos de red vulnerables.

Por el contrario, en arquitecturas distribuidas orientadas a la nube o integradas en plataformas empresariales, el estándar prescribe el uso de *Server-Sent Events* (SSE) sobre el protocolo HTTP. Este modelo bidireccional y asíncrono permite escalar los servidores independientemente de las aplicaciones cliente, soportando despliegues modernos basados en contenedores y facilitando la integración con pasarelas de API corporativas. Independientemente del transporte físico seleccionado, la mensajería subyacente se rige invariablemente por el protocolo de llamada a procedimiento remoto JSON-RPC 2.0, lo que asegura una serialización de datos uniforme y una compatibilidad universal entre todos los agentes del ecosistema.

4.4. Resiliencia, Seguridad y Gobernanza

La integración de sistemas corporativos con agentes autónomos generativos exige la implementación de mecanismos de defensa profundos dictados desde el propio diseño de la arquitectura. El principio de mínimo privilegio es fundamental en este contexto; dicta que el servidor MCP debe operar en un entorno de ejecución aislado (*sandbox*), poseyendo acceso exclusivo y limitado a las bases de datos o sistemas de archivos estrictamente necesarios para su dominio funcional. Esta segmentación perimetral previene de forma efectiva que una vulnerabilidad en la validación de una herramienta concreta se traduzca en una escalada de privilegios dentro de la red interna de la organización.

Paralelamente, la gobernanza operativa del sistema requiere una gestión de excepciones altamente estructurada. Ante peticiones malformadas por parte del modelo o la indisponibilidad temporal de los recursos subyacentes, el servidor MCP está obligado a emitir códigos de error estandarizados y practicar una degradación elegante del servicio (*graceful degradation*). Este enfoque técnico permite a la inteligencia artificial interpretar el fallo de manera coherente para reformular su estrategia operativa o, en su defecto, informar a la aplicación anfitriona de manera transparente, garantizando así la estabilidad, la trazabilidad y la auditabilidad ininterrumpida de toda la arquitectura de software.

5. Estándares ISO aplicados

En el despliegue de sistemas donde agentes autónomos (IA) interactúan con activos críticos de una empresa, la arquitectura debe estar validada por estándares internacionales. Esto asegura que el sistema sea **auditable, seguro y escalable**.

5.1. ISO/IEC 25010: Ingeniería de Calidad del Producto de Software

Esta norma define un modelo de calidad que evalúa el software en ocho características principales. Al implementar un **Servidor MCP**, impactamos directamente en las más críticas para sistemas distribuidos:

A. Mantenibilidad y Modularidad

La ISO 25010 exige que el software sea modificado con el mínimo riesgo.

- **Relación con MCP:** El protocolo aplica el patrón de **Inversión de Dependencias**. Al separar la "capa cognitiva" (el LLM) de la "capa de ejecución" (el servidor MCP), cualquier cambio en la base de datos o en la API externa solo afecta al servidor MCP. El modelo de IA permanece agnóstico, lo que reduce el coste de mantenimiento y evita la regresión de errores en el núcleo del sistema.

B. Interoperabilidad

Mide la capacidad de los sistemas para intercambiar información de forma coherente.

- **Relación con MCP:** Tradicionalmente, integrar un nuevo modelo de IA requería reescribir todos los conectores de datos (complejidad $O(N * M)$). MCP estandariza este intercambio mediante JSON-RPC 2.0. Cumplir con ISO 25010 en este punto significa que tu arquitectura es capaz de "enchufar y usar" (Plug & Play) cualquier herramienta o modelo que respete el estándar, garantizando una vida útil del software mucho mayor.

5.2. ISO/IEC 42001: El Primer Estándar Mundial de Gestión de IA

Esta es la norma definitiva para las organizaciones que desarrollan o utilizan sistemas de IA. Su objetivo es gestionar los riesgos y las oportunidades asociados a la IA de forma sistemática.

A. Trazabilidad y Rendición de Cuentas (Accountability)

La norma exige que cada acción realizada por un sistema de IA sea rastreable.

- **El Servidor MCP como "Caja Negra" de Auditoría:** En una integración directa, es difícil saber qué envió la IA a la base de datos. Con MCP, el servidor actúa como un **proxy de seguridad**. Cada vez que la IA usa una herramienta (**Tool Use**), el servidor MCP registra:
 1. El prompt que originó la llamada.
 2. Los argumentos JSON generados por la IA.
 3. La respuesta exacta del sistema externo.
- Esto cumple con los requisitos de **transparencia** de la ISO 42001, permitiendo auditorías forenses en caso de que la IA realice una acción no deseada.

B. Gestión de Riesgos y Seguridad del Contexto

ISO 42001 enfatiza la protección de datos y la mitigación de alucinaciones o acciones maliciosas.

- **Control Perimetral:** El servidor MCP retiene las credenciales reales (claves de API, contraseñas de DB). La IA nunca "ve" las llaves del reino; solo ve las herramientas que el servidor le expone semánticamente. Esto crea un entorno de **Privilegio Mínimo**, un requisito fundamental para la certificación en esta norma.

5.3. Sinergia Técnica: Por qué MCP es el vehículo para la certificación ISO

La importancia de unir MCP con estas ISO radica en la **confianza sistémica**. Sin un protocolo estándar como MCP, cumplir con las normativas ISO en un entorno de microservicios e IA sería una pesadilla burocrática y técnica.

1. **Uniformidad:** MCP proporciona la estructura técnica (handshake, recursos, herramientas) que la ISO 25010 pide para demostrar que un sistema es robusto.
2. **Validación de Datos:** El uso de **JSON Schema** en MCP para validar lo que la IA quiere hacer es un control técnico directo que satisface las exigencias de "precisión y robustez" de la ISO 42001.
3. **Desacoplamiento Operativo:** Permite que los auditores de calidad revisen la "capa de herramientas" (Servidor MCP) de forma independiente a la "capa de razonamiento", simplificando los procesos de certificación internacional.

6. Bibliografía

- ❖ **AENOR.** (s. f.). *Certificación de Sistemas de Gestión de Inteligencia Artificial (ISO/IEC 42001)*. Recuperado de: <https://www.aenor.com/certificacion/empresas/tecnologias-de-la-informacion/gestion-inteligencia-artificial>
- ❖ **Amazon Web Services.** (s. f.). *¿Qué es la arquitectura orientada a servicios (SOA)?* Recuperado de: <https://aws.amazon.com/es/what-is/service-oriented-architecture/>
- ❖ **Anthropic.** (2024). *Model Context Protocol (MCP) Official Documentation*. Recuperado de la documentación técnica del estándar MCP.
- ❖ **Farley, D.** (2023). *How to use Specs in Software Engineering* [Archivo de Vídeo]. Material de referencia sobre la metodología de especificación técnica.
- ❖ **ISO 25000.** (s. f.). *ISO/IEC 25010: Product Quality Models*. Recuperado de: <https://iso25000.com/index.php/normas-iso-25000/iso-25010>
- ❖ **O'Reilly Media.** (2024). *Building Intelligent Systems with LLMs and MCP*.
- ❖ **Richardson, L., & Ruby, S.** (2007). *RESTful Web Services*. O'Reilly Media.
- ❖ **Wikipedia.** (s. f.). *Enterprise service bus (ESB)*. Recuperado de: https://es.wikipedia.org/wiki/Enterprise_service_bus
- ❖ **[Material Audiovisual].** *MCP: La nueva Revolución IA*. Material de estudio de la asignatura.